

INFORMÁTICA

[Área personal](#) / [Mis cursos](#) / [\(29185\) INFORMÁTICA](#) / [Tema 6. Divide y vencerás](#) / [Lab 6. Divide y vencerás](#)

Lab 6. Divide y vencerás

Resuelve todos los ejercicios en la misma entrega. Esta página realizará unas pruebas básicas y no permitirá entregas con errores sintácticos o con demasiados fallos de funcionamiento. En varios ejercicios se piden algoritmos de cierta complejidad. Tómallo como una sugerencia, para mejorar tus habilidades de programación. El sistema de pruebas no comprobará si se cumple el criterio de complejidad.

1. Vuelta en monedas

Implementar una función `monedas(valor)` que recibe como argumento una cantidad monetaria (número real). Debe devolver la cantidad de monedas de cada tipo que deben usarse para componer esa cantidad con el mínimo posible de monedas.

Se emplearán monedas de 1, 2, 5, 10, 20 y 50 céntimos y de 1 y 2 euros.

Ejemplo:

```
>>> monedas(7.48)
((3,2),(1,1),(2,0.2),(1,0.05),(1,0.02),(1,0.01))
```

2. Ordenación indirecta

Se tiene un vector `v` de n elementos del que se quiere obtener información como si estuviese ordenado, pero que no puede ordenarse por motivos de trabajo (tampoco pueden realizarse copias en memoria del vector). Para poder hacerlo, se quiere crear un vector de índices `Ind`, de n elementos de tal manera que el valor `Ind[i]` sea precisamente la posición del vector original con el dato que debería estar en la posición `i` si efectivamente estuviese ordenado.

Ejemplo:

```
>>> ordenar_indirecto((50, 98, 10, 63, 31, 25, 63, 74))
(2, 5, 4, 0, 3, 6, 7, 1)
```

El primer elemento (2) es la posición de `v` donde está 10, que es el elemento que quedaría primero si se ordenase `v`, el segundo (5) es la posición de 25, que sería el segundo elemento en el vector ordenado, etc.

Nota: Este ejercicio está tomado de [esta página](#). Para su resolución se propone el algoritmo de *merge sort*, que está explicado en [esta página](#).

3. Media de un vector

Usando la técnica divide y vencerás, resolver el problema de calcular la media de un vector de n enteros, siendo n potencia de 2.

Definir la función `media(v)` que implementa el algoritmo desarrollado.

Nota: Ejercicio extraído de [esta página](#).

4. Subintervalo mayor

Implementa la función `subintervalo_mayor` que recibe como argumento una secuencia de intervalos **cerrados**, expresados como tuplas `(a,b)`. La función debe determinar el subintervalo más grande resultante de la unión de dichos intervalos. Ver los ejemplos para más detalles.

Ejemplo:

```
>>> subintervalo_mayor(((1,4),(5,6)))
(1,4)
>>> subintervalo_mayor(((1,5),(4,6)))
(1,6)
```

5. Cuadrado más grande

Se trata de resolver el problema de encontrar el rectángulo de ceros más grande en una tabla cuadrada de números. Programar una función `rectangulo_maximo` para resolver el problema por divide y vencerás. Dicha función recibe la tabla como una tupla de tuplas. Se asume que la tabla es cuadrada y las dimensiones son un múltiplo de 2. El resultado debe representarse con dos tuplas correspondientes a

las coordenadas de la esquina superior izquierda (menor x e y) e inferior derecha (mayor x e y) respectivamente.

Ejemplo:

```
>>> rectangulo_maximo(((1,0,0,1),(1,0,0,1),(0,0,0,1),(1,1,1,1)))  
((1,0),(3,3))
```

Nota: Ejercicio extraído de [esta página](#).

6. Particionado

Implementa la función `particion(L,k)` que reparte los elementos de `L` en dos partes alrededor del elemento `L[k]` (pivote). Devuelve los menores que `L[k]`, seguidos de `L[k]` y posteriormente los mayores que `L[k]`. Se asume que todos los valores son diferentes.

```
>>> particion([32, 17, 41, 18, 52, 98, 24, 65], 2)  
[18, 17, 24, 32], 41, [65, 52, 98]
```

El pivote es el elemento que ocupa la posición 2 (41). El orden de los elementos menores que 41 no tiene importancia, pero todos deben estar antes del 41 en el resultado. Igualmente, el orden de los elementos mayores de 41 no tiene importancia, pero todos deben estar después del 41.

7. El problema de selección

Implementar la función `seleccionar_rapido(L,k)` que devuelve el elemento que ocuparía la posición `k` si la secuencia de números `L` estuviera ordenada. Las posiciones son relativas a 0. Es decir, la primera posición sería el valor `k=0`, la segunda `k=1`.

El problema de selección es un problema a medio camino entre ordenación y búsqueda, que puede resolverse aplicando *divide y vencerás* de forma muy eficiente en tiempo $O(n)$.

Para ello se sigue el procedimiento habitual:

1. Si hay solución inmediata, se devuelve. En este caso, la solución inmediata existe cuando `L` tiene un elemento. En dicho caso la solución es el propio elemento.
2. Si no hay solución inmediata se divide el problema. En este caso debe identificarse un *pivote* de manera aleatoria (ver `random.choice`) y determinar la lista de los menores que el pivote (`Lm`) y los mayores que el pivote (`Lm`).
 - o Si la longitud de `Lm` es mayor que `k` entonces tenemos que aplicar nuevamente el algoritmo en `Lm` con el mismo `k`.
 - o Si la longitud de `Lm` es exactamente `k` entonces hemos tenido suerte, el pivote es la respuesta.
 - o En caso contrario el pivote está entre los elementos de `Lm`. Repetimos el algoritmo en `Lm` restando a `k` la longitud de `Lm`.
3. La solución global es directamente la solución al subproblema identificado en el paso anterior.

8. Los dos mayores

Diseñar un algoritmo para calcular el mayor y el segundo mayor elemento de un array de enteros, utilizando la técnica divide y vencerás. Diseña la función `max2(L)` que implementa dicho algoritmo.

Ejemplo:

```
>>> max2([4, 2, 1, 3])  
(4, 3)
```

Nota: Ejercicio extraído de [esta página](#).

9. Majority report

Dado un array de n enteros `E`, se dice que un entero es un elemento mayoritario de `E` si aparece más de $n/2$ veces en `E`. Implementar una función `mayoritario(E)` que lo encuentre en un tiempo lineal. Si no existe debe elevar una excepción `ValueError`.

Nota: Ejercicio extraído de [esta página](#).

10. Buscar sumandos

La función `buscar_sumandos(V,x)` recibe un vector `V` de n números reales y un real `x`. Se quiere determinar si existen dos índices i,j en tales que `V[i] + V[j] == x`. Busca un algoritmo cuya complejidad sea $O(n \log(n))$. Si no existen debe elevar la excepción `ValueError`.

Nota: Ejercicio extraído de [esta página](#).

Pruebas

A continuación se incluyen las 10 pruebas que será necesario pasar para aceptar esta entrega. Consulta [esta página](#) para aprender cómo pueden ayudarte las pruebas a desarrollar tu propio programa.

```
# Descomenta la siguiente línea y la última para ejecutar las pruebas
#from unittest import TestCase, main
```

```
class Test(TestCase):
    def test_monedas(self):
        self.assertEqual(monedas(9.4), ((4, 2), (1, 1), (2, 0.2)))
        self.assertEqual(monedas(1.5), ((1, 1), (1, 0.5)))
        self.assertEqual(monedas(.01), ((1, 0.01),))
        self.assertEqual(monedas(0), tuple())

    def test_ordenar_indirecto(self):
        self.assertEqual(ordenar_indirecto((50, 98, 10, 63, 31, 25, 63, 74)),
            (2, 5, 4, 0, 3, 6, 7, 1))
        self.assertEqual(ordenar_indirecto(tuple(range(1,10))), tuple(range(9)))
        self.assertEqual(ordenar_indirecto(tuple()), tuple())
        self.assertEqual(ordenar_indirecto((1,)), (0,))

    def test_media(self):
        self.assertEqual(media([1,2,4,8,16,32,64,128]), 31.875)
        self.assertEqual(media([2]), 2)
        self.assertEqual(media([]),0)
        self.assertEqual(media([1,5]), 3)

    def test_subintervalo_mayor(self):
        self.assertEqual(subintervalo_mayor(((1,4),(5,6))), (1,4))
        self.assertEqual(subintervalo_mayor(((1,5),(4,6))), (1,6))
        self.assertEqual(subintervalo_mayor(((5,7),(9,11),(2,5),(1,4),(4,6))), (1,7))
        self.assertEqual(subintervalo_mayor(((4,6))), (4,6))

    def test_rectangulo_maximo(self):
        self.assertEqual(rectangulo_maximo(((1,0,0,1),(1,0,0,1),(0,0,0,1),(1,1,1,1))),
            ((1,0),(3,3)))
        self.assertEqual(rectangulo_maximo(((1,0),(1,0))), (1,0),(2,2)))
        self.assertEqual(rectangulo_maximo(((0,)),), ((0,0),(1,1)))
        self.assertEqual(rectangulo_maximo(((0,)),), ((0,0),(1,1)))

    def test_particion(self):
        def tp(L,k):
            a,b,c = particion(L,k)
            self.assertEqual(len(a) + len(c), len(L) - 1)
            self.assertTrue(all(e < b for e in a))
            self.assertTrue(all(e > b for e in c))
            self.assertEqual(b, L[k])
            tp([32, 17, 41, 18, 52, 98, 24, 65], 2)
            tp([52, 98, 24], 2)
            tp([52, 1, 98], 2)
            tp([52, 5, 1], 2)

        def test_seleccionar_rapido(self):
            self.assertEqual(seleccionar_rapido([5], 0), 5)
            from random import shuffle
            L = list(range(8))
            self.assertEqual(seleccionar_rapido(L, 3), 3)
            L = list(range(3,18))
            shuffle(L)
            self.assertEqual(seleccionar_rapido(L,3), 6)
            L = list(range(4,100,2))
            shuffle(L)
            self.assertEqual(seleccionar_rapido(L,5), 14)

    def test_max2(self):
        self.assertEqual(max2([1,2]), (2,1))
        from random import shuffle
        L = list(range(8))
        self.assertEqual(max2(L), (7,6))
        L = list(range(3,18))
        shuffle(L)
        self.assertEqual(max2(L), (17,16))
        L = list(range(4,100,2))
        shuffle(L)
        self.assertEqual(max2(L), (98,96))

    def test_mayoritario(self):
        self.assertEqual(mayoritario([1,1,2]), 1)
        self.assertEqual(mayoritario([1,2,2,1,3,1,1]), 1)
        self.assertEqual(mayoritario([1]), 1)
```

mayoritario([1,2,3])

```
def test_buscar_sumandos(self):
    def ts(V,x):
        i,j = buscar_sumandos(V, x)
        self.assertEqual(i,j)
        self.assertEqual(V[i]+V[j], x)
    ts([12, 4, 14, 17, 9], 13)
    ts([1, 1], 2)
    ts([1, 2, 1], 2)
    with self.assertRaises(ValueError):
        buscar_sumandos([1,2,3], 8)

# Si usas Jupyter descomenta la última línea
# Si usas IDLE, Python o PyCharm descomenta la penúltima
# main()
# main(argv=['first-arg-is-ignored'], exit=False)
```

Estado de la entrega

Número del intento	Este es el intento 1.
Estado de la entrega	No entregado
Estado de la calificación	Sin calificar
Fecha de entrega	sábado, 15 de junio de 2019, 00:00:00
Tiempo restante	184 días 4 horas
Última modificación	-

Agregar entrega

Todavía no has realizado una entrega

◀ Tema 6. Divide y vencerás

Ir a...

Ejemplos de divide y vencerás ▶

Usted se ha identificado como PABLO MANUEL MARTÍN ISABEL (Salir)
(29185) INFORMÁTICA

Español - Internacional (es)
English (en)

Español - Internacional (es)

Descargar la app para dispositivos móviles